

Atividade: simular concorrência, bloqueios, espera e inconsistências em transações

Etapa 1. Criar o banco de teste

```
DROP TABLE IF EXISTS contas;
```

```
CREATE TABLE contas (  
  id INT PRIMARY KEY,  
  titular VARCHAR(100),  
  saldo DECIMAL(10,2)  
);
```

```
INSERT INTO contas (id, titular, saldo) VALUES  
(1, 'Ana', 1000.00),  
(2, 'Bruno', 500.00),  
(3, 'Carlos', 300.00),  
(4, 'Daniela', 800.00);
```

```
SELECT * FROM contas;
```

Pergunta 1

Qual é a finalidade de manter dados iniciais conhecidos antes dos testes de concorrência?

Pergunta 2

Por que é importante que a tabela esteja em um estado consistente antes do início dos experimentos?

P1: Manter dados iniciais conhecidos garante **reprodutibilidade e controle** nos testes. Como todos os experimentos partem do mesmo estado (Ana = R\$1.000, Bruno = R\$500, etc.), qualquer alteração nos saldos após os testes pode ser atribuída com certeza à concorrência e não a valores aleatórios ou pré-existentes.

Além disso, facilita a **validação matemática** dos resultados: se Ana transfere R\$ 200 para Bruno, o resultado esperado é previsível. Qualquer valor diferente indica uma anomalia como lost update ou dirty read.

P2: Se a tabela já começar com inconsistências (saldo negativo, chave duplicada, etc.), qualquer problema encontrado durante os testes não pode ser atribuído com segurança à concorrência pode ser apenas herança de um estado anterior corrompido.

Começar consistente também reflete melhor a realidade de produção, onde os bancos operam normalmente antes de sofrerem acessos simultâneos. Assim, os fenômenos simulados, como phantom reads ou dirty reads são comprovadamente causados pelas transações concorrentes, tornando o experimento confiável e os resultados válidos.

Etapa 2. Testar bloqueio com FOR UPDATE

Abra duas sessões.

Sessão 1

```
START TRANSACTION;
```

```
SELECT * FROM contas  
WHERE id = 1  
FOR UPDATE;
```

```
UPDATE contas  
SET saldo = saldo - 100  
WHERE id = 1;
```

Não execute `COMMIT` ainda.

Sessão 2

```
START TRANSACTION;
```

```
UPDATE contas  
SET saldo = saldo + 50  
WHERE id = 1;
```

Agora volte para a Sessão 1 e execute:

```
COMMIT;
```

Depois finalize a Sessão 2 com:

```
COMMIT;
```

Pergunta 3

O que aconteceu com a operação realizada na Sessão 2?

Pergunta 4

Por que a segunda sessão precisou aguardar?

Pergunta 5

Qual é a função do comando `FOR UPDATE` nesse experimento?

```
docker-compose.yml MySQL Docker.session.sql X
C: > Users > sophy > OneDrive > Documentos > bd2 > C:\Users\sophy\OneDrive\Documentos\bd2\MySQL Docker.session.sql

1 DROP TABLE IF EXISTS contas;
2
3 CREATE TABLE contas (
4     id INT PRIMARY KEY,
5     titular VARCHAR(100),
6     saldo DECIMAL(10,2)
7 );
8
9 INSERT INTO contas (id, titular, saldo) VALUES
10 (1, 'Ana', 1000.00),
11 (2, 'Bruno', 500.00),
12 (3, 'Carlos', 300.00),
13 (4, 'Daniela', 800.00);
14
15 SELECT * FROM contas;

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\sophy\OneDrive\Documentos\bd2> docker exec -it meu-mysql mysql -u root --password=root123
Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> USE meuBANCO;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> UPDATE contas SET saldo = saldo + 50 WHERE id = 1;
```

```
docker-compose.yml MySQL Docker.session.sql X
C: > Users > sophy > OneDrive > Documentos > bd2 > C:\Users\sophy\OneDrive\Documentos\bd2\MySQL Docker.session.sql

1 DROP TABLE IF EXISTS contas;
2
3 CREATE TABLE contas (
4     id INT PRIMARY KEY,
5     titular VARCHAR(100),
6     saldo DECIMAL(10,2)
7 );
8
9 INSERT INTO contas (id, titular, saldo) VALUES
10 (1, 'Ana', 1000.00),
11 (2, 'Bruno', 500.00),
12 (3, 'Carlos', 300.00),
13 (4, 'Daniela', 800.00);
14
15 SELECT * FROM contas;

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\sophy\OneDrive\Documentos\bd2> docker exec -it meu-mysql mysql -u root --password=root123

mysql>
mysql> SELECT * FROM contas WHERE id = 1 FOR UPDATE;
+----+-----+-----+
| id | titular | saldo |
+----+-----+-----+
| 1 | Ana | 1000.00 |
+----+-----+-----+
1 row in set (0.00 sec)

mysql>
mysql> UPDATE contas SET saldo = saldo - 100 WHERE id = 1;
Query OK, 1 row affected (0.01 sec)
Rows matched: 1 Changed: 1 Warnings: 0

mysql> COMMIT;
Query OK, 0 rows affected (0.01 sec)

mysql>
```

```
1 DROP TABLE IF EXISTS contas;
2
3 CREATE TABLE contas (
4     id INT PRIMARY KEY,
5     titular VARCHAR(100),
6     saldo DECIMAL(10,2)
7 );
8
9 INSERT INTO contas (id, titular, saldo) VALUES
10 (1, 'Ana', 1000.00),
11 (2, 'Bruno', 500.00),
12 (3, 'Carlos', 300.00),
13 (4, 'Daniela', 800.00);
14
15 SELECT * FROM contas;
```

```
PS C:\Users\sophy\OneDrive\Documents\bd2> docker exec -it meu-mysql mysql -u root --password=root123

mysql>
mysql> UPDATE contas SET saldo = saldo + 50 WHERE id = 1;
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql>
mysql> COMMIT;
Query OK, 0 rows affected (0.00 sec)

mysql> SELECT * FROM contas WHERE id = 1;
+----+-----+-----+
| id | titular | saldo |
+----+-----+-----+
| 1  | Ana    | 950.00 |
+----+-----+-----+
1 row in set (0.00 sec)

mysql>
```

P3: A Sessão 2 ficou bloqueada ao tentar executar o UPDATE na linha de id = 1, pois a Sessão 1 já havia adquirido um bloqueio exclusivo sobre ela. Como a Sessão 1 demorou para dar COMMIT, o MySQL atingiu o tempo limite de espera e retornou o erro Lock wait timeout exceeded. Após o COMMIT da Sessão 1, a Sessão 2 conseguiu executar sua operação normalmente, resultando no saldo final de R\$ 950,00

P4: Porque a Sessão 1 detinha um bloqueio exclusivo sobre a linha de id = 1, adquirido pelo FOR UPDATE. O InnoDB não permite que duas transações modifiquem a mesma linha simultaneamente, então a Sessão 2 ficou em fila de espera até a Sessão 1 finalizar. Isso é exatamente o mecanismo de controle de concorrência do banco funcionando para evitar inconsistências.

P5: O FOR UPDATE bloqueia explicitamente as linhas retornadas pelo SELECT, sinalizando que a transação pretende modificá-las. Isso impede que outras sessões alterem essas linhas até que o COMMIT seja dado. Sem ele, a Sessão 2 poderia ler e modificar o saldo ao mesmo tempo que a Sessão 1, causando um lost update onde uma das alterações seria perdida sem qualquer aviso de erro.

Etapa 3. Testar acesso concorrente a registros diferentes

Abra duas sessões.

Sessão 1

```
START TRANSACTION;
```

```
UPDATE contas
```

```
SET saldo = saldo - 50  
WHERE id = 1;
```

Sessão 2

```
START TRANSACTION;
```

```
UPDATE contas  
SET saldo = saldo + 70  
WHERE id = 4;
```

Finalize ambas com:

```
COMMIT;
```

Depois consulte:

```
SELECT * FROM contas;
```

Pergunta 6

Por que, nesse caso, as duas transações tendem a coexistir sem espera significativa?

Pergunta 7

O que esse comportamento revela sobre bloqueios em nível de linha?



```
PS C:\Users\sophy\OneDrive\Documentos\bd2> docker exec -it meu-mysql mysql -u root --password=root123  
mysql> UPDATE contas SET saldo = saldo + 70 WHERE id = 4;  
Query OK, 1 row affected (0.01 sec)  
Rows matched: 1 Changed: 1 Warnings: 0  
  
mysql> COMMIT;  
Query OK, 0 rows affected (0.01 sec)  
  
mysql> SELECT * FROM contas;  
+----+-----+-----+  
| id | titular | saldo |  
+----+-----+-----+  
| 1  | Ana     | 900.00 |  
| 2  | Bruno   | 500.00 |  
| 3  | Carlos  | 300.00 |  
| 4  | Daniela | 870.00 |  
+----+-----+-----+  
4 rows in set (0.01 sec)  
  
mysql>
```

P6: Porque cada sessão está modificando uma linha diferente, Sessão 1 atualiza id = 1 (Ana) e Sessão 2 atualiza id = 4 (Daniela). O InnoDB trabalha com bloqueio em nível de linha, ou seja, só bloqueia a linha específica que está sendo alterada, não a tabela inteira. Como não há disputa pela mesma linha, as duas transações rodam em paralelo sem interferência.

P7: Revela que o bloqueio em nível de linha é muito mais eficiente do que bloqueio de tabela. Apenas a linha envolvida na transação fica bloqueada, permitindo que outras sessões acessem e modifiquem linhas diferentes simultaneamente. Isso aumenta muito a

concorrência e o desempenho em sistemas com muitos acessos paralelos, já que as transações só se bloqueiam mutuamente quando disputam exatamente o mesmo registro.

Etapa 4. Testar leitura durante transação não finalizada

Sessão 1

```
START TRANSACTION;
```

```
UPDATE contas  
SET saldo = saldo - 200  
WHERE id = 2;
```

Sem confirmar ainda.

Sessão 2

```
SELECT * FROM contas WHERE id = 2;
```

Depois volte para a Sessão 1 e execute:

```
ROLLBACK;
```

Pergunta 8

Qual era o objetivo de consultar o mesmo registro em outra sessão antes do COMMIT?

Pergunta 9

Como esse experimento se relaciona com o conceito de isolamento?



The screenshot shows a terminal window with the following content:

```
PS C:\Users\sophy\OneDrive\Documentos\bd2> docker exec -it meu-mysql mysql -u root --password=root123
```

id	titular	saldo
1	Ana	900.00
2	Bruno	500.00
3	Carlos	300.00
4	Daniela	870.00

```
4 rows in set (0.01 sec)
```



```
mysql> SELECT * FROM contas WHERE id = 2;
```

id	titular	saldo
2	Bruno	500.00

```
1 row in set (0.01 sec)
```



```
mysql>
```

The terminal window has a status bar at the bottom showing: Ln 15, Col 22, Spaces: 4, UTF-8, CRLF, () SQL.

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS C:\Users\sophy\OneDrive\Documentos\bd2> docker exec -it meu-mysql mysql -u root --password=root123

mysql> UPDATE contas SET saldo = saldo - 50 WHERE id = 1;
Query OK, 1 row affected (0.05 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> COMMIT;
Query OK, 0 rows affected (0.00 sec)

mysql> START TRANSACTION;
Query OK, 0 rows affected (0.01 sec)

mysql> UPDATE contas SET saldo = saldo - 200 WHERE id = 2;
Query OK, 1 row affected (0.02 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> ROLLBACK;
Query OK, 0 rows affected (0.02 sec)

mysql> 
```

P8: O objetivo era verificar se a Sessão 2 conseguia ler o valor modificado pela Sessão 1 antes de ela confirmar a transação. Isso testa se o banco permite dirty read, leitura de dados "sujos", ou seja, alterações que ainda não foram confirmadas e podem ser desfeitas. No MySQL com InnoDB, o nível de isolamento padrão é REPEATABLE READ, então a Sessão 2 enxerga o valor original de Bruno (R\$ 500,00), ignorando a alteração não confirmada da Sessão 1.

P9: Esse experimento demonstra na prática o isolamento entre transações. O isolamento define o quanto uma transação pode "ver" o que outras estão fazendo antes de finalizarem. No nível REPEATABLE READ (padrão do MySQL), cada transação trabalha com sua própria visão consistente dos dados, sem enxergar alterações não confirmadas de outras sessões. O ROLLBACK da Sessão 1 reforça isso, qualquer leitura do valor intermediário seria inválida, pois a alteração foi desfeita. O isolamento existe justamente para proteger contra esse tipo de inconsistência.

Etapa 5. Testar repetição de leitura

Sessão 1

```
START TRANSACTION;
```

```
SELECT * FROM contas WHERE id = 3;
```

Sessão 2

```
START TRANSACTION;
```

```
UPDATE contas
SET saldo = saldo + 100
WHERE id = 3;
```

```
COMMIT;
```

Agora volte para a Sessão 1 e repita:

```
SELECT * FROM contas WHERE id = 3;
```

Finalize a Sessão 1:

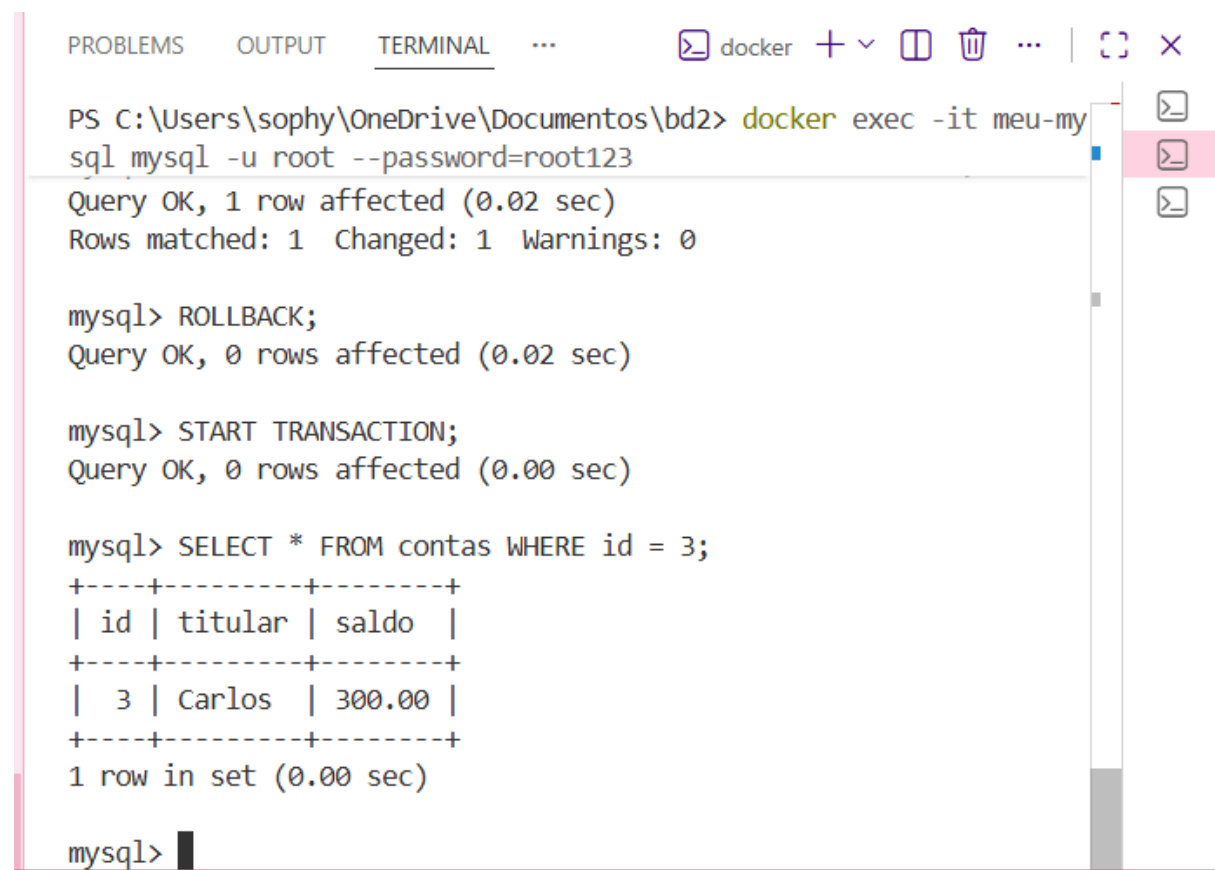
```
COMMIT;
```

Pergunta 10

O valor lido na Sessão 1 permaneceu o mesmo ou mudou?

Pergunta 11

Que tipo de fenômeno esse teste procura identificar?



```
PROBLEMS OUTPUT TERMINAL ... docker + v [icon] [icon] ... [icon] [icon]
PS C:\Users\sophy\OneDrive\Documentos\bd2> docker exec -it meu-my
sql mysql -u root --password=root123
Query OK, 1 row affected (0.02 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> ROLLBACK;
Query OK, 0 rows affected (0.02 sec)

mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

mysql> SELECT * FROM contas WHERE id = 3;
+----+-----+-----+
| id | titular | saldo |
+----+-----+-----+
| 3 | Carlos | 300.00 |
+----+-----+-----+
1 row in set (0.00 sec)

mysql> 
```



```
PROBLEMS OUTPUT TERMINAL ... docker + v [ ] [ ] ... | [ ] [ ] [ ] X

PS C:\Users\sophy\OneDrive\Documentos\bd2> docker exec -it meu-mysql mysql -u root --password=root123

mysql> SELECT * FROM contas WHERE id = 3;
+----+-----+-----+
| id | titular | saldo |
+----+-----+-----+
| 3  | Carlos  | 300.00 |
+----+-----+-----+
1 row in set (0.00 sec)

mysql> SELECT * FROM contas WHERE id = 3;
+----+-----+-----+
| id | titular | saldo |
+----+-----+-----+
| 3  | Carlos  | 300.00 |
+----+-----+-----+
1 row in set (0.00 sec)

mysql> 
```

P10: O valor permaneceu o mesmo nas duas leituras da Sessão 1. Mesmo após a Sessão 2 ter atualizado e confirmado o saldo de Carlos (id = 3), a Sessão 1 continuou enxergando o valor original (R\$ 300,00) nas duas consultas. Isso acontece porque o MySQL no nível REPEATABLE READ garante que uma transação sempre veja a mesma versão dos dados durante toda a sua execução, independente de commits feitos por outras sessões.

P11: Esse teste procura identificar o fenômeno chamado Non-Repeatable Read (leitura não repetível) que ocorre quando uma transação lê o mesmo registro duas vezes e obtém valores diferentes porque outra transação modificou e confirmou esse registro no intervalo entre as leituras. O experimento mostra que o MySQL previne esse fenômeno no nível REPEATABLE READ, pois a Sessão 1 enxergou o mesmo valor de Carlos nas duas leituras, mesmo com o COMMIT da Sessão 2 ocorrendo no meio.

Etapa 6. Simular atualização concorrente sobre o mesmo dado

Abra duas sessões.

Sessão 1

```
START TRANSACTION;
```

```
SELECT * FROM contas WHERE id = 4;
```

```
UPDATE contas  
SET saldo = saldo - 100  
WHERE id = 4;
```

Sessão 2

```
START TRANSACTION;
```

```
SELECT * FROM contas WHERE id = 4;
```

```
UPDATE contas  
SET saldo = saldo - 200  
WHERE id = 4;
```

Finalize ambas com `COMMIT`, observando a ordem de execução e depois consulte:

```
SELECT * FROM contas WHERE id = 4;
```

Pergunta 12

Por que operações concorrentes sobre o mesmo registro exigem maior controle?

Pergunta 13

Que inconsistência pode surgir quando duas transações tentam atualizar o mesmo dado quase ao mesmo tempo?

PROBLEMS OUTPUT TERMINAL ...  docker + v   ... |  

```
PS C:\Users\sophy\OneDrive\Documentos\bd2> docker exec -it meu-mysql mysql -u root --password=root123
```

```
Rows matched: 1  Changed: 1  Warnings: 0
```

```
mysql> COMMIT;
Query OK, 0 rows affected (0.01 sec)
```

```
mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)
```

```
mysql> SELECT * FROM contas WHERE id = 4;
```

```
+----+-----+-----+
| id | titular | saldo |
+----+-----+-----+
|  4 | Daniela | 870.00 |
+----+-----+-----+
1 row in set (0.00 sec)
```

```
mysql> UPDATE contas SET saldo = saldo - 200 WHERE id = 4;
```

PROBLEMS OUTPUT TERMINAL ...  docker + v   ... |  

```
PS C:\Users\sophy\OneDrive\Documentos\bd2> docker exec -it meu-mysql mysql -u root --password=root123
```

```
1 row in set (0.00 sec)
```

```
mysql> UPDATE contas SET saldo = saldo - 200 WHERE id = 4;
Query OK, 1 row affected (32.54 sec)
Rows matched: 1  Changed: 1  Warnings: 0
```

```
mysql> COMMIT;
Query OK, 0 rows affected (0.01 sec)
```

```
mysql> SELECT * FROM contas WHERE id = 4;
```

```
+----+-----+-----+
| id | titular | saldo |
+----+-----+-----+
|  4 | Daniela | 570.00 |
+----+-----+-----+
1 row in set (0.00 sec)
```

```
mysql>
```

P12: Porque duas transações atualizando o mesmo dado simultaneamente podem gerar resultados incorretos. Cada sessão lê o valor atual, calcula a alteração e grava o resultado mas se as duas leituras acontecerem antes de qualquer gravação, ambas partem do mesmo valor base e uma das atualizações pode sobrescrever a outra. Por isso o banco precisa serializar o acesso, garantindo que uma transação espere a outra terminar antes de modificar o mesmo registro.

P13: O fenômeno é chamado de Lost Update (atualização perdida). Acontece quando duas transações leem o mesmo valor (ex: Daniela com R\$870,00), calculam suas alterações de forma independente e a segunda gravação sobrescreve a primeira fazendo com que uma das atualizações seja completamente perdida. Nesse experimento, o InnoDB evita isso através do bloqueio de linha: a Sessão 2 fica bloqueada até a Sessão 1 confirmar, garantindo que o saldo final reflita ambas as operações corretamente ($870 - 100 - 200 = \text{R\$}570,00$).

Etapa 7. Testar espera por lock

Sessão 1

```
START TRANSACTION;
```

```
SELECT * FROM contas WHERE id = 2 FOR UPDATE;
```

Mantenha a transação aberta.

Sessão 2

```
START TRANSACTION;
```

```
UPDATE contas
```

```
SET saldo = saldo + 10
```

```
WHERE id = 2;
```

Agora, depois de observar a espera, volte para a Sessão 1 e execute:

```
COMMIT;
```

Pergunta 14

Qual evidência mostra que havia um bloqueio ativo sobre o registro?

Pergunta 15

Por que a liberação do lock depende do fim da transação?

PROBLEMS OUTPUT TERMINAL ...

 docker + v   ... |  

```
PS C:\Users\sophy\OneDrive\Documentos\bd2> docker exec -it meu-my  
sql mysql -u root --password=root123
```

```
Query OK, 1 row affected (32.54 sec)  
Rows matched: 1 Changed: 1 Warnings: 0
```

```
mysql> COMMIT;  
Query OK, 0 rows affected (0.01 sec)
```

```
mysql> SELECT * FROM contas WHERE id = 4;  
+----+-----+-----+  
| id | titular | saldo |  
+----+-----+-----+  
| 4 | Daniela | 570.00 |  
+----+-----+-----+  
1 row in set (0.00 sec)
```

```
mysql> START TRANSACTION;  
Query OK, 0 rows affected (0.00 sec)
```

```
mysql> UPDATE contas SET saldo = saldo + 10 WHERE id = 2;
```

PROBLEMS OUTPUT TERMINAL ...

 docker + v   ... |  

```
PS C:\Users\sophy\OneDrive\Documentos\bd2> docker exec -it meu-my  
sql mysql -u root --password=root123
```

```
mysql> COMMIT;  
Query OK, 0 rows affected (0.02 sec)
```

```
mysql> START TRANSACTION;  
Query OK, 0 rows affected (0.00 sec)
```

```
mysql> SELECT * FROM contas WHERE id = 2 FOR UPDATE;  
+----+-----+-----+  
| id | titular | saldo |  
+----+-----+-----+  
| 2 | Bruno | 500.00 |  
+----+-----+-----+  
1 row in set (0.01 sec)
```

```
mysql> COMMIT;  
Query OK, 0 rows affected (0.01 sec)
```

```
mysql>
```

Ln 15, Col 22 Spaces: 4 UTF-8 CRLF {} SQL  

P14: A evidência é o terminal da Sessão 2 travando sem retorno após o UPDATE o cursor fica piscando sem apresentar Query OK, indicando que está aguardando. Isso prova que a linha de id = 2 estava bloqueada pela Sessão 1. Quando o COMMIT é dado na Sessão 1, a Sessão 2 imediatamente recebe o retorno e conclui sua operação confirmando que o bloqueio foi liberado nesse momento.

P15: Porque enquanto a transação está aberta, o banco não tem como saber se ela vai confirmar ou cancelar as alterações. Liberar o bloqueio antes do COMMIT ou ROLLBACK permitiria que outra sessão modificasse o mesmo dado, corrompendo a operação em andamento. O lock só é liberado ao final da transação justamente para garantir que o trabalho seja concluído de forma atômica e isolada, princípios fundamentais do ACID.

Etapa 8. Testar bloqueio com duas leituras de atualização

Sessão 1

```
START TRANSACTION;
```

```
SELECT * FROM contas WHERE id = 1 FOR UPDATE;
```

Sessão 2

```
START TRANSACTION;
```

```
SELECT * FROM contas WHERE id = 1 FOR UPDATE;
```

Depois finalize a Sessão 1 com:

```
COMMIT;
```

Pergunta 16

Por que a segunda leitura com `FOR UPDATE` não pôde prosseguir imediatamente?

Pergunta 17

Em que essa situação difere de uma consulta `SELECT` comum?

PROBLEMS OUTPUT TERMINAL ...

 docker + v   ... |  

```
PS C:\Users\sophy\OneDrive\Documentos\bd2> docker exec -it meu-mysql mysql -u root --password=root123
```

```
| id | titular | saldo |
+----+-----+-----+
| 4 | Daniela | 570.00 |
+----+-----+-----+
1 row in set (0.00 sec)
```

```
mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)
```

```
mysql> UPDATE contas SET saldo = saldo + 10 WHERE id = 2;START TRANSACTION;
Query OK, 1 row affected (0.01 sec)
Rows matched: 1 Changed: 1 Warnings: 0
```

```
Query OK, 0 rows affected (0.02 sec)
```

```
mysql> SELECT * FROM contas WHERE id = 1 FOR UPDATE;
```

Ln 15, Col 22 Spaces: 4 UTF-8 CRLF { } SQL  

PROBLEMS OUTPUT TERMINAL ...

 docker + v   ... |  

```
PS C:\Users\sophy\OneDrive\Documentos\bd2> docker exec -it meu-mysql mysql -u root --password=root123
```

```
mysql> COMMIT;
Query OK, 0 rows affected (0.01 sec)
```

```
mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)
```

```
mysql> SELECT * FROM contas WHERE id = 1 FOR UPDATE;
+----+-----+-----+
| id | titular | saldo |
+----+-----+-----+
| 1 | Ana      | 900.00 |
+----+-----+-----+
1 row in set (0.00 sec)
```

```
mysql> COMMIT;
Query OK, 0 rows affected (0.00 sec)
```

```
mysql>
```

Ln 15, Col 22 Spaces: 4 UTF-8 CRLF { } SQL  

P16: Porque o FOR UPDATE da Sessão 1 já havia adquirido um bloqueio exclusivo sobre a linha de id = 1. O FOR UPDATE da Sessão 2 também exige exclusividade e dois bloqueios exclusivos sobre a mesma linha não podem coexistir. Diferente de leituras comuns que podem acontecer em paralelo, qualquer intenção de escrita exige acesso único, então a Sessão 2 ficou bloqueada até a Sessão 1 liberar com o COMMIT.

Etapa 9. Simular risco de atualização perdida

Considere o seguinte cenário conceitual:

- saldo atual da conta 1 = 1000
- Transação A lê saldo 1000 e decide subtrair 100
- Transação B lê saldo 1000 e decide subtrair 200
- A grava 900
- B grava 800

Pergunta 18

Qual seria o saldo correto ao final, caso ambas as operações fossem consideradas corretamente?

Pergunta 19

Por que o resultado 800 caracteriza uma atualização perdida?

P18: Se ambas as operações fossem aplicadas corretamente e em sequência, o saldo final deveria ser R\$700,00 (1000 - 100 - 200). Isso só acontece quando a segunda transação lê o valor já atualizado pela primeira, ou seja, B leria 900 e subtrairia 200, gravando 700.

P19: Porque a Transação B leu o saldo original (1000) antes de A gravar sua alteração, calculou sua subtração com base nesse valor desatualizado e sobrescreveu o resultado de A. Com isso, a subtração de 100 feita por A foi completamente ignorada, como se nunca tivesse acontecido. O banco registrou 800 quando o correto seria 700, ou seja, R\$100,00 simplesmente desapareceram do sistema sem nenhum erro sendo apontado. Esse é o perigo do lost update: a inconsistência passa despercebida e os dados ficam corrompidos silenciosamente.

Etapa 10. Testar inserções concorrentes em outra tabela

Crie a tabela:

```
DROP TABLE IF EXISTS log_operacoes;
```

```
CREATE TABLE log_operacoes (  
  id INT AUTO INCREMENT PRIMARY KEY,  
  descricao VARCHAR(200)  
);
```


Abra duas sessões.

Sessão 1

```
START TRANSACTION;
```

```
INSERT INTO log_operacoes (descricao)  
VALUES ('Operacao realizada pela sessao 1');
```

Sessão 2

```
START TRANSACTION;
```

```
INSERT INTO log_operacoes (descricao)  
VALUES ('Operacao realizada pela sessao 2');
```

Finalize ambas com `COMMIT` e consulte:

```
SELECT * FROM log_operacoes;
```

Pergunta 20

Por que inserções em linhas diferentes nem sempre geram conflito direto?

Pergunta 21

O que esse experimento mostra sobre concorrência quando não há disputa pelo mesmo registro?

```
PROBLEMS  OUTPUT  TERMINAL  ...  docker + -  [ ] [ ] ... | [ ] [ ] [ ] X

PS C:\Users\sophy\OneDrive\Documentos\bd2> docker exec -it meu-mysql mysql -u root --password=root123

mysql> INSERT INTO log_operacoes (descricao) VALUES ('Operacao realizada pela sessao 1');
Query OK, 1 row affected (0.01 sec)

mysql> COMMIT;
Query OK, 0 rows affected (0.00 sec)

mysql> SELECT * FROM log_operacoes;
+----+-----+
| id | descricao |
+----+-----+
|  1 | Operacao realizada pela sessao 1 |
|  2 | Operacao realizada pela sessao 2 |
+----+-----+
2 rows in set (0.00 sec)

mysql> 
```

Ln 15, Col 22 Spaces: 4 UTF-8 CRLF { } SQL [] [] []

P20: Porque cada INSERT cria uma linha nova e independente na tabela, não há disputa por um registro já existente. O bloqueio em nível de linha só entra em conflito quando duas transações tentam acessar a mesma linha. Como cada sessão insere seu próprio registro com id diferente (gerado pelo AUTO_INCREMENT), elas não interferem uma na outra e podem executar em paralelo sem espera.

P21: Mostra que o banco de dados é capaz de processar múltiplas transações simultaneamente de forma eficiente quando não há sobreposição de registros. A concorrência só se torna um problema quando duas transações disputam o mesmo recurso. Quando cada transação trabalha em seu próprio espaço, o InnoDB permite execução paralela sem bloqueios, o que é o comportamento ideal em sistemas com alto volume de inserções simultâneas.

Etapa 11. Simular bloqueio prolongado

Sessão 1

```
START TRANSACTION;
```

```
SELECT * FROM contas WHERE id = 3 FOR UPDATE;
```

Não finalize imediatamente.

Sessão 2

```
START TRANSACTION;
```

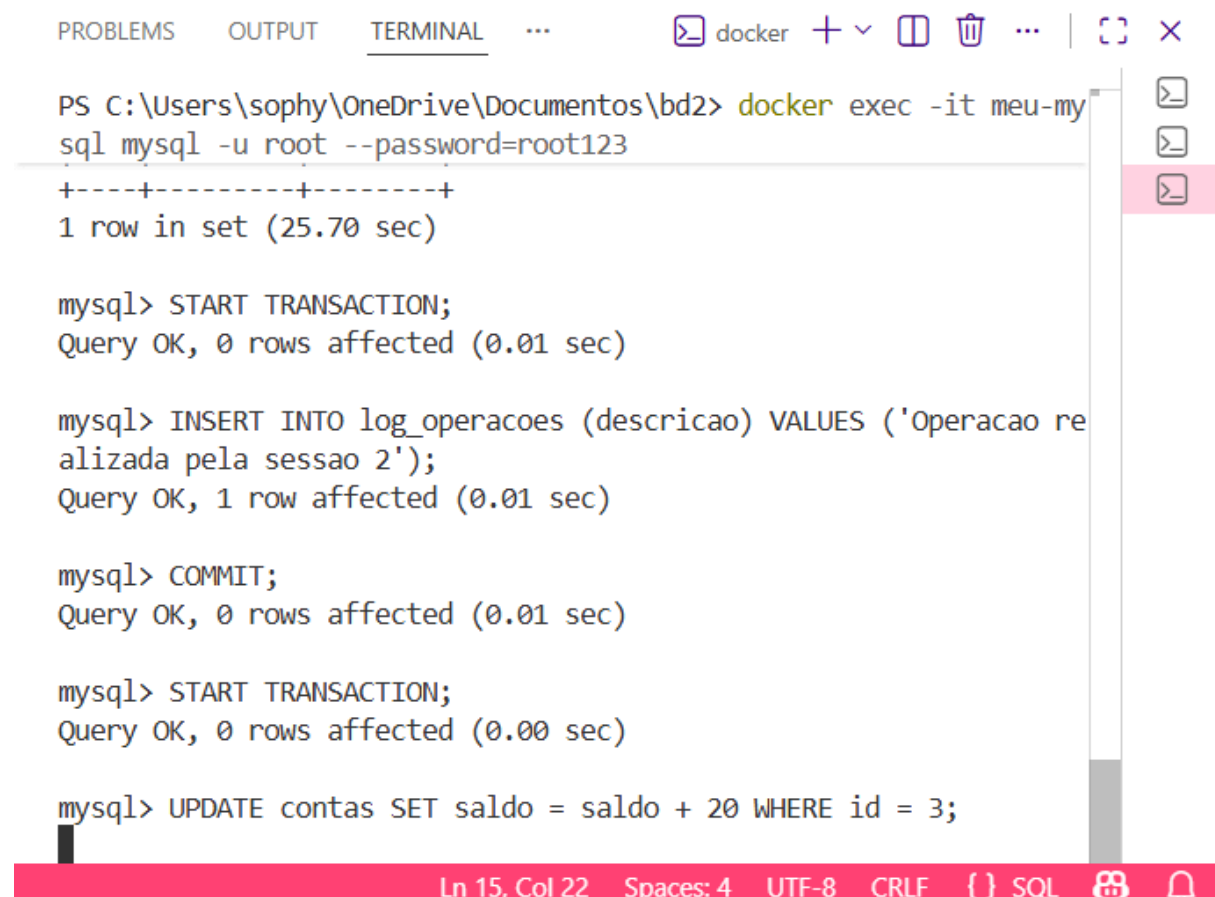
```
UPDATE contas  
SET saldo = saldo + 20  
WHERE id = 3;
```

Pergunta 22

Quais impactos um bloqueio mantido por muito tempo pode causar em um sistema real?

Pergunta 23

Por que transações longas tendem a ser indesejáveis em ambientes concorrentes?



```
PROBLEMS OUTPUT TERMINAL ... docker + v [icon] [icon] ... | [icon] [icon] X
```

```
PS C:\Users\sophy\OneDrive\Documentos\bd2> docker exec -it meu-mysql  
sql mysql -u root --password=root123
```

```
+---+-----+-----+  
1 row in set (25.70 sec)
```

```
mysql> START TRANSACTION;  
Query OK, 0 rows affected (0.01 sec)
```

```
mysql> INSERT INTO log_operacoes (descricao) VALUES ('Operacao re  
alizada pela sessao 2');  
Query OK, 1 row affected (0.01 sec)
```

```
mysql> COMMIT;  
Query OK, 0 rows affected (0.01 sec)
```

```
mysql> START TRANSACTION;  
Query OK, 0 rows affected (0.00 sec)
```

```
mysql> UPDATE contas SET saldo = saldo + 20 WHERE id = 3;
```

```
Ln 15, Col 22 Spaces: 4 UTF-8 CRLF { } SQL [bug icon] [bell icon]
```


Pergunta 24

Como verificar se o banco permaneceu consistente após todos os cenários executados?

Pergunta 25

Por que a análise final dos dados é importante após testes de concorrência

```
PROBLEMS  OUTPUT  TERMINAL  ...  docker + v [icon] [icon] ... | [icon] [icon] X

PS C:\Users\sophy\OneDrive\Documentos\bd2> docker exec -it meu-my
sql mysql -u root --password=root123

+----+-----+-----+
| 1 | Ana   | 900.00 |
| 2 | Bruno | 510.00 |
| 3 | Carlos| 400.00 |
| 4 | Daniela| 570.00 |
+----+-----+-----+
4 rows in set (0.00 sec)

mysql> SELECT * FROM log_operacoes;
+----+-----+-----+
| id | descricao                                     |
+----+-----+-----+
| 1 | Operacao realizada pela sessao 1 |
| 2 | Operacao realizada pela sessao 2 |
+----+-----+-----+
2 rows in set (0.00 sec)

mysql> s
```

Ln 15, Col 22 Spaces: 4 UTF-8 CRLF {} SQL [icon] [icon]

P24: A verificação é feita comparando o estado final com o estado inicial conhecido, somando todas as operações realizadas ao longo dos experimentos. Se os saldos finais batem matematicamente com todas as adições e subtrações executadas nas etapas anteriores, o banco permaneceu consistente. Além disso, a presença dos dois registros em log_operacoes confirma que as inserções concorrentes foram gravadas corretamente. Qualquer valor inesperado indicaria que alguma transação foi aplicada incorretamente ou perdida.

P25: Porque os testes de concorrência simulam situações de alto risco para a integridade dos dados. A análise final confirma que os mecanismos de bloqueio, isolamento e controle de transações funcionaram corretamente em todos os cenários impedindo lost updates, dirty reads e inconsistências. Sem essa verificação, um erro silencioso poderia passar despercebido. É também uma forma de documentar que o banco se comportou conforme o esperado, o que é essencial tanto em ambientes acadêmicos quanto em produção.

7. Atividade dissertativa

Questão 26 — O que é concorrência em banco de dados?

Concorrência ocorre quando múltiplos usuários ou processos acessam e manipulam o banco de dados ao mesmo tempo. É algo natural em sistemas reais, mas exige controle para que as operações simultâneas não interfiram umas nas outras e corrompam os dados.

Questão 27 — Papel dos bloqueios no controle de concorrência

Os bloqueios impedem que duas transações modifiquem o mesmo dado simultaneamente. Quando uma transação bloqueia um registro, as demais precisam aguardar sua liberação, garantindo que as alterações sejam aplicadas de forma ordenada e segura.

Questão 28 — Registros iguais vs. registros diferentes

Quando duas transações acessam o mesmo registro, há disputa e uma precisa esperar a outra terminar. Quando acessam registros diferentes, não há conflito e ambas executam em paralelo sem interferência, aproveitando melhor o desempenho do banco.

Questão 29 — Importância do FOR UPDATE

O FOR UPDATE bloqueia explicitamente as linhas selecionadas, sinalizando que a transação pretende modificá-las. Sem ele, outra sessão poderia alterar o mesmo registro entre o SELECT e o UPDATE, gerando inconsistências como atualização perdida.

Questão 30 — Transação esperando outra liberar recurso

Significa que a transação está em fila de espera porque o registro que ela precisa acessar está bloqueado por outra transação ainda em aberto. Ela só consegue prosseguir após o COMMIT ou ROLLBACK da transação que detém o bloqueio.

Questão 31 — Conceito de atualização perdida

Ocorre quando duas transações leem o mesmo valor, calculam alterações de forma independente e a segunda gravação sobrescreve a primeira. O resultado da transação A é perdido como se nunca tivesse acontecido, gerando dados incorretos sem nenhum erro aparente.

Questão 32 — Por que o isolamento é essencial em sistemas multiusuário

O isolamento garante que cada transação trabalhe com uma visão consistente dos dados, sem ser afetada por operações não confirmadas de outras sessões. Sem ele, leituras sujas, atualizações perdidas e inconsistências seriam frequentes em ambientes com muitos acessos simultâneos.

Questão 33 — Como uma leitura pode ser afetada por outra transação

Se o banco permitisse dirty read, uma transação poderia ler um valor modificado por outra que ainda não confirmou. Se essa outra der ROLLBACK, o valor lido nunca existiu de fato, gerando decisões baseadas em dados inválidos.

Questão 34 — Por que transações longas prejudicam o desempenho

Transações longas mantêm bloqueios por mais tempo, impedindo que outras operações avancem. Isso cria filas de espera, aumenta o risco de timeout e deadlock, e consome mais memória com logs de desfazer, degradando o desempenho geral do sistema.

Questão 35 — Relação entre concorrência e consistência

Concorrência sem controle compromete a consistência, pois operações simultâneas podem gerar resultados incorretos. Os mecanismos de bloqueio e isolamento existem justamente para permitir alta concorrência sem abrir mão da integridade dos dados.

Questão 36 — Exemplo real de disputa pelo mesmo dado

Em um sistema bancário, dois caixas eletrônicos processando saques simultâneos da mesma conta disputam o mesmo saldo. Sem controle, ambos poderiam ler o saldo original e liberar valores que juntos ultrapassam o limite disponível.

Questão 37 — Por que nem toda operação simultânea gera conflito

Porque o conflito só ocorre quando duas transações disputam o mesmo registro. Operações em registros diferentes são independentes e executam em paralelo sem interferência, como demonstrado na Etapa 3 com Ana e Daniela sendo atualizadas simultaneamente.

Questão 38 — Como o banco impede que alterações simultâneas corrompam os dados

Através de bloqueios em nível de linha, controle de isolamento e o protocolo ACID. O banco serializa o acesso aos mesmos registros, garante que alterações não confirmadas fiquem invisíveis para outras sessões e desfaz operações incompletas em caso de falha.

Questão 39 — Sistema bancário sem mecanismos de lock

Seria um caos. Dois saques simultâneos poderiam aprovar valores que juntos superam o saldo. Transferências poderiam ser aplicadas parcialmente. Saldos seriam sobrescritos sem considerar operações paralelas, gerando prejuízos financeiros e dados completamente inconsistentes.

Questão 40 — Importância de observar a ordem de execução

A ordem de execução define qual transação adquire o bloqueio primeiro e como as demais reagem. Observar essa ordem nos testes práticos permite entender como o banco serializa operações concorrentes e verificar se o resultado final é o esperado matematicamente.

9. Questão integradora

Questão 41

Considerando todos os experimentos realizados, explique de forma integrada como concorrência, bloqueios e isolamento atuam juntos para evitar inconsistências no banco de dados.

P41: A concorrência é uma realidade inevitável em sistemas reais, múltiplos usuários acessam e modificam os mesmos dados ao mesmo tempo. Por si só, ela representa um risco à integridade do banco, pois operações simultâneas sobre os mesmos registros podem gerar resultados incorretos. Os experimentos realizados demonstraram na prática

como o banco lida com esse desafio através de dois mecanismos principais: bloqueios e isolamento.

Os bloqueios atuam como guardiões do acesso aos dados. Como visto nas etapas 2, 7, 8 e 11, quando uma transação adquire um bloqueio sobre um registro, seja através do FOR UPDATE ou de um UPDATE direto, outras transações que tentam acessar o mesmo dado ficam em espera. Isso garante que as alterações sejam aplicadas de forma ordenada, uma por vez, evitando o lost update demonstrado conceitualmente na Etapa 9, onde duas transações partindo do mesmo valor base poderiam fazer uma sobrescrever a outra silenciosamente.

O isolamento, por sua vez, controla o que cada transação consegue enxergar das demais enquanto ainda estão em andamento. Como demonstrado na Etapa 4, o MySQL no nível REPEATABLE READ impede que uma sessão leia dados modificados por outra que ainda não confirmou, protegendo contra o dirty read. Já na Etapa 5, ficou evidente que o banco garante leituras consistentes dentro de uma mesma transação, mesmo que outra sessão confirme alterações no intervalo entre duas consultas idênticas.

Juntos, esses mecanismos formam uma camada de proteção completa: os bloqueios controlam quem pode escrever e quando, enquanto o isolamento controla o que cada transação pode ver. A Etapa 3 mostrou ainda que esse sistema é inteligente o suficiente para permitir paralelismo quando não há disputa, transações em registros diferentes executam simultaneamente sem bloqueios desnecessários, preservando o desempenho.

Em resumo, concorrência, bloqueios e isolamento não atuam de forma independente, são camadas complementares que permitem ao banco de dados atender múltiplos usuários simultaneamente sem abrir mão da consistência e integridade dos dados.